Contenedores `list`, `deque`, `stack` y `queue`

Lo que ya construyeron, con el nombre de la biblioteca

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Octubre 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Usar `list` y `deque` con sus iteradores, y reconocer en cada operación la lista doblemente enlazada o los bloques que la respaldan (OA2, OA3).
- Dar el costo en tiempo y en espacio de cada operación de `list`, `deque`, `stack` y `queue`, y explicar de dónde sale (OA1, OA2).
- Explicar qué contenedor guardan por dentro `stack` y `queue` y cuándo se puede cambiar (OA3).
- Escoger el contenedor de un programa a partir de las operaciones que hace, y contrastar la elección con tiempos medidos (OA1, OA5).

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Cinco implementaciones a mano I

Construida	Qué la hace rápida	Qué le cuesta
Lista doble	`anterior` en cada nodo: desenlazar en $\Theta(1)$	llegar a la posición p, $\Theta(p)$
Cola doble	lista doble con cabeza y `ultimo`: los cuatro extremos en $\Theta(1)$	leer el elemento i, $\Theta(i)$
Pila sobre nodos	la cima es la cabeza	nada: todo $\Theta(1)$
Cola sobre nodos	entra por `ultimo`, sale por la cabeza	nada: todo $\Theta(1)$
Cola sobre arreglo circular	dos índices que dan la vuelta	capacidad fija

Quicksort cerró el tema de ordenamiento: `particionar` deja el pivote en su casilla en $\Theta(n)$, y el algoritmo cuesta $\Theta(n \log n)$ con particiones balanceadas y $\Theta(n^2)$ en el peor caso.

Cinco implementaciones a mano II

Pregunta de hoy

La biblioteca trae `list`, `deque`, `stack` y `queue`. ¿Cuál de las cinco de la tabla es cada una?

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Un reproductor con cursor I

Problema

Un reproductor guarda una lista de canciones y un cursor. Llegan 100 000 órdenes: insertar una canción donde está el cursor, borrar la del cursor, mover el cursor una canción adelante o atrás. ¿Qué cuesta con un `vector`?

Un reproductor con cursor II

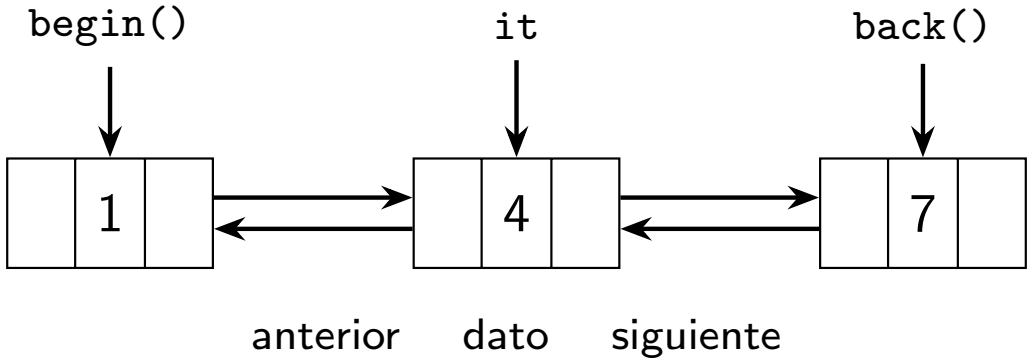
Con `vector`

`insert` y `erase` en la posición p corren los $n - p$ elementos que siguen. Si el cursor anda cerca del comienzo, cada orden cuesta del orden de n, y las 100 000 órdenes suman del orden de n^2.

Lo que pide el problema

Insertar y borrar donde ya se está parado, sin correr a nadie, y moverse un paso en cualquier dirección. Eso lo hace la lista doblemente enlazada: con el nodo en la mano, insertar y desenlazar tocan cuatro punteros, y `anterior` permite retroceder.

`list<T>`: la lista doble con nombres nuevos I



Lista doble del curso	`list<T>`
`insertar(0, e)`, `agregar(e)`	`push_front(e)`, `push_back(e)`
`eliminar(0)`, `eliminar(n-1)`	`pop_front()`, `pop_back()`
insertar antes de un nodo	`insert(it, e)`
`desenlazar(x)` y `delete`	`erase(it)`
`tamano`, `vacia`	`size()`, `empty()`

El iterador de `list` hace lo que hacía el puntero al nodo: `++it` sigue a `siguiente` y `--it` sigue a `anterior`. No hay `l[i]` ni `it + 3`.

Insertar y borrar con el iterador I

```
int main() {
  list<int> l;
  l.push_back(4);
  l.push_back(7);
  l.push_front(1);
  imprimir(l);                          // 1 4 7
  conIterador(l);
  l.pop_front();
  l.pop_back();
  imprimir(l);                          // 9
  printf("size: %d\n", (int) l.size());
  return 0;
}
```

Insertar y borrar con el iterador II

```
// insertar y borrar donde senala el iterador
void conIterador(list<int> &l) {
  list<int>::iterator it = l.begin();
  ++it;                                  // senala el 4
  l.insert(it, 9);                       // entra antes del 4
  imprimir(l);                           // 1 9 4 7
  printf("*it = %d\n", *it);             // sigue en el 4
  it = l.erase(it);                      // devuelve el siguiente: el 7
  printf("*it = %d\n", *it);
  --it;                                  // retrocede al 9
  printf("*it = %d\n", *it);
}
```

Insertar y borrar con el iterador III

Pregunta

Tras `l.insert(it, 9)`, ¿a quién señala `it`? En el `vector`, un iterador guardado antes de `insert` dejaba de valer.

Salida

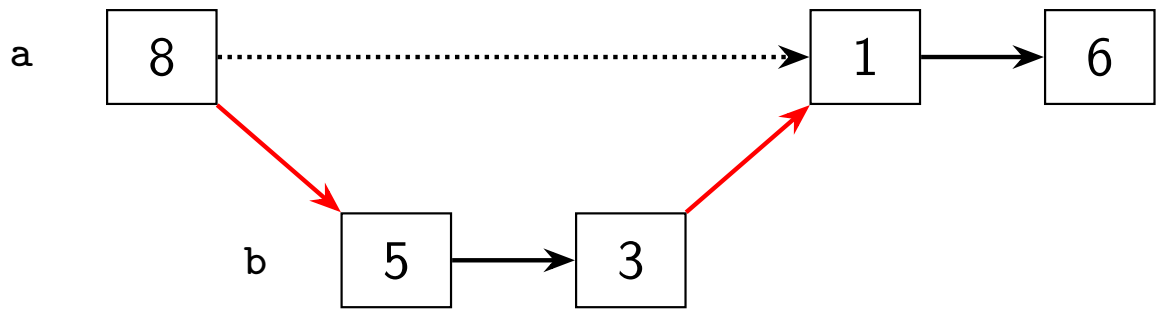
`1 4 7`, después `1 9 4 7`, y `*it = 4`: `it` sigue en el 4. Tras `erase`, `*it = 7`; tras `--it`, `*it = 9`. Con los dos `pop` queda `9` y `size: 1`.

- El 4 sigue en su nodo: insertar creó un nodo nuevo y reescribió dos punteros de los vecinos, sin mover a nadie. Por eso `insert` no invalida ningún iterador de `list`.
- `erase(it)` invalida solo los iteradores al nodo borrado, y devuelve el que sigue, igual que en `vector`.

`splice`: mover nodos sin copiarlos I

Problema

Se tiene `a = <8 1 6>` y `b = <5 3>`. Hay que dejar los elementos de `b` dentro de `a`, antes del 1. Con `insert` se copian uno por uno y `b` sigue llena. ¿Hace falta copiar?



Los dos enlaces rojos y sus dos `anterior` reemplazan al punteado: cuatro punteros, sin importar cuántos nodos traiga `b`.

splice: mover nodos sin copiarlos II

```
int main() {
  list<int> a = {8, 1, 6};
  list<int> b = {5, 3};
  list<int>::iterator it = a.begin();
  ++it;                              // senala el 1
  a.splice(it, b);                   // b entra antes del 1
  imprimir("a", a);                  // 8 5 3 1 6
  imprimir("b", b);                  // vacia
  printf("*it = %d\n", *it);         // sigue en el 1

  a.sort();
  imprimir("a", a);                  // 1 3 5 6 8
  printf("*it = %d\n", *it);         // el 1, ahora al frente
  return 0;
}
```

`splice`: mover nodos sin copiarlos III

Salida

`a: 8 5 3 1 6`, `b:` vacía, `*it = 1`. Después de `sort`, `a: 1 3 5 6 8` y `*it = 1`: el iterador sigue en el mismo nodo, que ahora está al frente.

- `splice(it, b)` con la lista entera cuesta $\Theta(1)$ y deja `b` vacía. Ningún nodo cambia de dirección, así que ningún iterador se invalida.

Ordenar una `list` I

Pregunta

Con un `vector` se ordenaba con `sort(v.begin(), v.end())`. ¿Qué pasa con una `list`?

```
list<int> l = {8, 1, 6};
std::sort(l.begin(), l.end());
```

Ordenar una `list` II

No compila

```
error: no match for ‘operator-’ (operand types are
‘std::_List_iterator<int>’ and ‘std::_List_iterator<int>’)
```

- `sort` calcula distancias y salta posiciones: `fin - ini`, `ini + k`. En el arreglo eso es aritmética; en la lista, llegar a la posición k es caminar k nodos, y la biblioteca no ofrece el salto.
- La lista trae su propio `l.sort()`: el merge sort sobre la lista enlazada, el que divide con dos punteros a distinta velocidad y combina empalmando nodos. Hace del orden de $n \log n$ comparaciones y no copia datos: reescribe enlaces. Por eso el `it` del ejemplo anterior seguía valiendo.

Lo que cobra `list` I

Operación	Tiempo	Por qué
`push_front`, `push_back`	$\Theta(1)$	se cuelga el nodo de la cabeza o del último y se reescribe el `anterior` o el `siguiente` del vecino
`pop_front`, `pop_back`	$\Theta(1)$	el vecino pasa a ser el extremo
`insert(it, e)`	$\Theta(1)$	dos punteros del nodo nuevo, el `siguiente` del de antes y el `anterior` del de `it`
`erase(it)`	$\Theta(1)$	el `siguiente` del anterior y el `anterior` del siguiente
`splice(it, b)`, `b` entera	$\Theta(1)$	dos enlaces de ida y dos de vuelta
`front`, `back`, `size`, `empty`	$\Theta(1)$	se leen la cabeza, el último y el contador
llegar al elemento p	$\Theta(p)$	`++it` de nodo en nodo
`l.sort()`	$\Theta(n \log n)$	merge sort que empalma nodos

Lo que cobra `list` II

Espacio

La lista ocupa $\Theta(n)$: dato y dos punteros por nodo, más lo que pide cada `new`. Ninguna operación de la tabla pide memoria aparte que crezca con n; `l.sort()` en la biblioteca de GCC usa 64 listas auxiliares vacías, $\Theta(1)$.

Lo que `list` no tiene

- Ni `l[i]` ni `it + k`: el acceso por posición es $\Theta(p)$, y la biblioteca no lo disfraza de operador.
- Ni `sort`, `binary_search` ni `lower_bound` de la biblioteca de algoritmos sirven como en el `vector`: los dos últimos compilan, pero cada salto a la mitad camina nodo a nodo.

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Lo que le faltaba a la cola doble I

Pregunta

La cola doble sobre la lista doble entra y sale por los dos extremos en $\Theta(1)$. Si además se pide leer el elemento i, ¿cuánto cuesta? ¿Y con un `vector`, cuánto cuesta entrar por el frente?

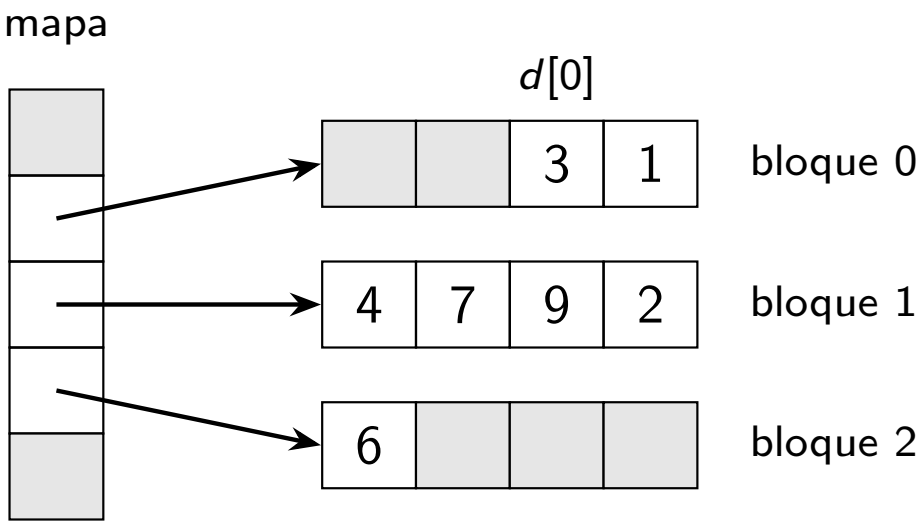
Lo que le faltaba a la cola doble II

	Entrar por el frente	Leer el elemento i
Cola doble sobre nodos	$\Theta(1)$	$\Theta(i)$: se camina
`vector`	$\Theta(n)$: se corre todo	$\Theta(1)$: se calcula
`deque`	$O(1)$	$O(1)$

La idea

Ni un solo arreglo ni un nodo por elemento: bloques de tamaño fijo, cada uno un arreglo pequeño, y un arreglo corto de punteros a los bloques, el *mapa*. Dentro de un bloque se calcula la casilla; entre bloques, el mapa dice dónde está cada uno.

Bloques y mapa I



El `deque <3 1 4 7 9 2 6>` con bloques de $B = 4$ casillas. El primer elemento está en la casilla 2 del bloque 0; sea $ini = 2$. Lo gris está reservado y vacío.

Bloques y mapa II

Leer $d[i]$

El elemento i está en el bloque $(ini + i)/B$, en la casilla $(ini + i)$ mód B. Para $d[5]$: $(2 + 5)/4 = 1$ y $(2 + 5)$ mód $4 = 3$, el 2. Son dos cuentas y dos accesos, el del mapa y el del bloque, sin importar i.

Entrar por el frente

`push_front(8)` escribe en la casilla 1 del bloque 0 y deja $ini = 1$. Cuando el primer bloque se llena, se pide uno nuevo y su puntero va en la casilla libre del mapa que está antes. Ningún elemento se corre.

- Si el mapa se queda sin casillas libres, se copia a uno más grande, como el `vector`. Se copian punteros a bloques, n/B de ellos, no los elementos.

Bloques y mapa III

- En la biblioteca de GCC cada bloque ocupa 512 bytes: 128 enteros.

deque<T>: la cola doble de la biblioteca I

Cola doble del curso	deque<T>
`insertarFrente`, `insertarFinal`	`push_front`, `push_back`
`eliminarFrente`, `eliminarFinal`	`pop_front`, `pop_back`
`frente`, `final`	`front`, `back`
no estaba	`d[i]`, `d.begin() + i`

deque<T>: la cola doble de la biblioteca II

```cpp
int main() {
  deque<int> d;
  d.push_back(4);
  d.push_back(7);
  d.push_front(1);
  d.push_front(3);
  imprimir(d);                          // 3 1 4 7
  printf("d[2] = %d\n", d[2]);
  printf("front %d, back %d\n", d.front(), d.back());

  d.pop_front();
  d.pop_back();
  imprimir(d);                          // 1 4
  d.insert(d.begin() + 1, 9);           // corre el lado corto
  imprimir(d);                          // 1 9 4
  printf("size: %d\n", (int) d.size());
  return 0;
}
```

deque<T>: la cola doble de la biblioteca III

Salida

`3 1 4 7`, `d[2] = 4`, `front 3, back 7`; tras los dos `pop`, `1 4`; tras el `insert`, `1 9 4` y `size: 3`.

Lo que cobra deque I

Operación	Tiempo	Por qué
`d[i]`, `front`, `back`	$O(1)$	bloque y casilla se calculan
`push_front`, `push_back`	$O(1)$	se escribe una casilla libre o se pide un bloque
`pop_front`, `pop_back`	$O(1)$	se mueve el extremo; un bloque vacío se libera
`insert`, `erase` en la posición p	$O(\text{mín}(p, n-p))$	se corre el lado más corto
`size`, `empty`	$O(1)$	se lee el contador

Espacio

$\Theta(n)$ más el mapa, con n/B punteros, y a lo sumo dos bloques a medio llenar. Un deque con un solo elemento ya reserva un bloque entero.

Lo que cobra `deque` II

Iteradores que dejan de valer

- `push_front` y `push_back` invalidan todos los iteradores, porque el mapa pudo mudarse. Los punteros y referencias a los elementos siguen valiendo: los bloques no se mueven.
- `insert` y `erase` en medio invalidan todo: se corrieron elementos.
- Tras cualquiera de ellas, el iterador se toma de nuevo, como en el `vector`.

`vector`, `deque` y `list`, lado a lado I

Operación	`vector`	`deque`	`list`
acceso por índice	$O(1)$	$O(1)$	no hay; caminar, $\Theta(p)$
entrar o salir por el final	$O(1)$ amortizado	$O(1)$	$\Theta(1)$
entrar o salir por el frente	$\Theta(n)$	$O(1)$	$\Theta(1)$
insertar o borrar en p	$O(n-p)$	$O(\text{mín}(p, n-p))$	$\Theta(1)$ con el iterador
`sort` de la biblioteca	sí	sí	no; `l.sort()`
`insert` invalida iteradores	sí	sí	no
memoria por elemento	el dato	el dato	dato y dos punteros

- La `list` es la única que inserta en medio sin correr a nadie, pero paga con no tener índice.
- El `deque` gana al `vector` por el frente y pierde poco en el índice: dos accesos en vez de uno.

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Un contrato encima de otro contenedor I

Pregunta

`stack` y `queue` ya se usaron con `push`, `pop`, `top` y `front`. ¿Dónde guardan los elementos? Si por dentro hay un contenedor con índice, ¿por qué no hay `s[2]`?

Un contrato encima de otro contenedor II

La cola sobre el TAD Lista ya hizo esto: una clase con una `Lista` adentro que solo deja pasar `encolar`, `desencolar` y `frente`. La biblioteca llama a eso un *adaptador*.

Adaptador	Por defecto	Se puede cambiar por	Lo que pide del de abajo
`stack<T>`	`deque<T>`	`vector<T>`, `list<T>`	`back`, `push_back`, `pop_back`
`queue<T>`	`deque<T>`	`list<T>`	`front`, `back`, `push_back`, `pop_front`

El contrato es lo que limita: la pila promete el último y nada más. Si dejara leer `s[2]`, ya no sería una pila, y el código que la usa podría depender de cómo está guardada por dentro.

Escoger lo que va debajo I

```
int main() {
  stack<int> a;                       // por debajo, un deque
  stack<int, vector<int> > b;         // por debajo, un vector
  queue<int, list<int> > c;           // por debajo, una list
  int x = 1;
  while (x <= 3) {
    a.push(x);
    b.push(10 * x);
    c.push(100 * x);
    x = x + 1;
  }
  printf("%d %d %d\n", a.top(), b.top(), c.front());
  a.pop();
  b.pop();
  c.pop();
  printf("%d %d %d\n", a.top(), b.top(), c.front());
  return 0;
}
```

Escoger lo que va debajo II

Salida

3 30 100 y, tras los tres pop, 2 20 200. Las pilas sacaron el último que entró y la cola el primero, sin importar qué contenedor guarda los datos.

Escoger lo que va debajo III

Pregunta

¿Y una `queue<int, vector<int> >`?

Compila hasta el primer pop

```
error: ‘class std::vector<int>’ has no member named ‘pop_front’
```

- El `vector` no trae `pop_front` porque le costaría $\Theta(n)$: correr todo un puesto. La biblioteca prefiere no ofrecer la operación a ofrecerla cara.
- Para la `stack`, el `vector` sí sirve: todo pasa por el final.

Lo que cobran los adaptadores I

Operación	Sobre `deque` o `list`	Sobre `vector`	Por qué
`push`	$O(1)$	$O(1)$ amortizado	un `push_back` del de abajo
`pop`	$O(1)$	$O(1)$	`pop_back` o `pop_front`
`top`, `front`, `back`	$O(1)$	$O(1)$	se lee un extremo
`size`, `empty`	$O(1)$	$O(1)$	se pregunta al de abajo

El espacio es el del contenedor de abajo: $\Theta(n)$. El adaptador no agrega nada.

Lo que cobran los adaptadores II

	Construida a mano	Biblioteca
Pila	`Pila` de `pila_nodos.h`: la cima es la cabeza	`stack<T>`
Cola	`Cola` de `cola_nodos.h`: entra por `ultimo`	`queue<T>`

Las dos construidas a mano guardan un nodo por elemento; las de la biblioteca, por defecto, bloques de un `deque`. Mismo contrato, mismos costos asintóticos, distinta memoria.

El tercer adaptador

`priority_queue` sale por el mayor, no por el más viejo. Va sobre un `vector` y llega con los montículos.

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Escoger por las operaciones I

Si el programa ...	Contenedor
lee por índice y crece por el final	`vector`
entra o sale por los dos extremos y lee por índice	`deque`
inserta y borra en medio con un cursor que se mueve paso a paso	`list`
mueve trozos enteros de una secuencia a otra	`list` con `splice`
solo usa el último que entró	`stack`
solo usa el más viejo	`queue`

Cuando dos sirven, se escoge el de contrato más estrecho: una `stack` dice al que lee el código lo que el programa hace, y un `deque` no lo dice.

Ejercicio en parejas: decidir I

Para cada programa, escojan el contenedor y digan qué cuesta la operación que más se repite.

1. Un navegador guarda las 50 últimas páginas visitadas. Cada visita entra al final; si ya hay 50, sale la más vieja. El botón "historial" muestra la página número i.
2. En una fila de espera, cualquiera puede retirarse desde la mitad. El programa guarda, por cada persona, dónde está en la fila, y la saca de ahí cuando se retira.
3. El botón "deshacer" de un editor de texto.
4. Se leen n notas y después se responden q consultas: ¿cuál es la nota del estudiante i?
5. Una impresora atiende los trabajos en el orden en que llegaron.
6. Dos filas de un banco se fusionan cuando cierra una caja: la segunda fila entera pasa detrás de la persona número 3 de la primera.

Ejercicio en parejas: decidir II

Lo que debe salir

1. `deque`: `push_back` y `pop_front` en $O(1)$ y la página i en $O(1)$. El `vector` pagaría $\Theta(n)$ en cada salida por el frente; la `list`, $\Theta(i)$ en cada consulta.
2. `list`: se guarda un iterador por persona, y `erase` la saca en $\Theta(1)$. Es la única de las tres en la que los iteradores guardados de los demás siguen valiendo tras borrar.
3. `stack`: solo se mira y se quita la última acción; todo $O(1)$.
4. `vector`: se llena una vez con `push_back` y cada consulta es $O(1)$.
5. `queue`: el más viejo sale primero; todo $O(1)$.
6. `list`: llegar a la persona 3 cuesta tres pasos y `splice` pega la otra fila en $\Theta(1)$, sin importar cuántos haya en ella.

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Lo que se midió I

Pregunta

`push_front` cuesta $O(1)$ en `deque` y en `list`, y `insert` al frente cuesta $\Theta(n)$ en `vector`. Si n se duplica, ¿cuánto debería crecer el tiempo de insertar n veces al frente en cada uno?

Lo que se midió II

```
void alFrenteVector(int n) {
  vector<int> v;
  int i = 0;
  while (i < n) {
    v.insert(v.begin(), i);
    i = i + 1;
  }
}

void alFrenteDeque(int n) {
  deque<int> d;
  int i = 0;
  while (i < n) {
    d.push_front(i);
    i = i + 1;
  }
}
```

`alFrenteList` es la misma, con `list<int>`.

Lo que se midió III

Para leer, `sumarDeque` suma `d[i]` con `i` de 0 a $n-1$. La `list` se suma de dos formas:

```
// el elemento i se alcanza caminando i pasos desde el comienzo
long sumarListPorIndice(list<int> &l) {
  long s = 0;
  int i = 0;
  while (i < (int) l.size()) {
    list<int>::iterator it = l.begin();
    int pasos = 0;
    while (pasos < i) {
      ++it;
      pasos = pasos + 1;
    }
    s = s + *it;
    i = i + 1;
  }
  return s;
}
```

Lo que se midió IV

```
long sumarListRecorrido(list<int> &l) {
  long s = 0;
  list<int>::iterator it = l.begin();
  while (it != l.end()) {
    s = s + *it;
    ++it;
  }
  return s;
}
```

Lo que se midió V

- El programa lee el reloj del procesador antes y después de cada llamada con `clock()`, de `<ctime>`, y divide la diferencia entre `CLOCKS_PER_SEC` para pasarla a segundos.
- Se compiló con `g++ -Wall -Wextra`, sin optimizar, en un AMD Ryzen 5 3600. Tres corridas difirieron en menos del 10 % en las cifras de más de una centésima.

Insertar n veces al frente I

Tiempos en segundos:

n	`vector`	`deque`	`list`
25 000	0,0184	0,0002	0,0027
50 000	0,0723	0,0004	0,0052
100 000	0,2931	0,0008	0,0101
200 000	1,2893	0,0016	0,0199

- Al duplicar n, el `vector` multiplica su tiempo por cerca de 4: son n inserciones que corren hasta n elementos, del orden de n^2.
- `deque` y `list` lo multiplican por 2: n operaciones de $O(1)$.
- Las dos son $O(1)$ por operación, pero con $n = 200\,000$ la `list` tarda unas 12 veces más que el `deque`: hace un `new` por elemento, y el `deque` uno cada 128 enteros.

Sumar los n elementos I

Tiempos en segundos:

n	`deque`, `d[i]`	`list`, por posición	`list`, iterador
5 000	0,0003	0,0384	0,0001
10 000	0,0005	0,1579	0,0003
20 000	0,0010	0,6366	0,0005
40 000	0,0020	2,5887	0,0006

- Leer la posición i de la `list` caminando desde el comienzo suma $0 + 1 + \cdots + (n - 1)$ pasos: al duplicar n, el tiempo se multiplica por 4. Con 40 000 elementos ya pasa de dos segundos.
- Recorrer la misma `list` con un iterador que avanza es $\Theta(n)$, y tarda menos que el `deque` con `d[i]`. El costo no está en la lista: está en volver a empezar desde `begin()` por cada elemento.

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Para el juez: UVa 11988 — Broken Keyboard I

UVa 11988 — Broken Keyboard (a.k.a. Beiju Text),
`https://onlinejudge.org/external/119/11988.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3139`

Qué pide

Cada línea es lo que se tecleó: entre 1 y 100 000 letras y guiones bajos, con dos caracteres especiales. `[` manda el cursor al inicio del texto (*Home*) y `]` al final (*End*). Hay que imprimir el texto que quedó en la pantalla. La entrada termina con el fin de archivo.

Entradas y salidas del ejemplo

`This_is_a_[Beiju]_text` deja `BeijuThis_is_a__text`, y `[[]][][]Happy_Birthday_to_Tsinghua_University` deja el texto sin corchetes.

Para el juez: UVa 11988 — Broken Keyboard II

Cómo atacarlo con lo de hoy

El texto es una `list<char>` y el cursor, un iterador. `[` lo pone en `begin()`, `]` en `end()`, y cada letra entra con `insert(cursor, c)`. `insert` pone la letra antes del cursor y el cursor no se mueve, así que la siguiente letra queda detrás de la anterior. Cada carácter cuesta $\Theta(1)$: $\Theta(n)$ por línea.

Las trampas

- Con `push_front` por cada letra después de un `[`, `[Beiju` sale `ujieB`: las letras tecleadas tras *Home* quedan en el orden en que se tecleó.
- Con un `vector`, cada letra corre lo que está detrás del cursor: del orden de n^2 en una línea de 100 000.
- Pueden venir corchetes seguidos y una línea que no deja letras.

Para el juez: UVa 12207 — That is Your Queue I

UVa 12207 — That is Your Queue,
`https://onlinejudge.org/external/122/12207.pdf`
Envío: `https://onlinejudge.org/index.php?option=com_onlinejudge&Itemid=25&page=submit_problem&problemid=3359`

Qué pide

Los ciudadanos $1, 2, \ldots, P$ hacen fila para el único hospital, con $1 \leq P \leq 10^9$. Llegan $C \leq 1000$ órdenes: `N` atiende al del frente, que vuelve a entrar al final, y `E x` pasa al ciudadano x al frente sin cambiar el orden de los demás. Por cada `N` se imprime a quién se atendió. Hay a lo sumo diez casos y la entrada termina con `0 0`.

Entradas y salidas del ejemplo

Con $P = 3$ y las órdenes `N N E 1 N N N`, salen `1 2 1 3 2`. Con $P = 10$ y dos `N`, salen `1 2`.

Para el juez: UVa 12207 — That is Your Queue II

Cómo atacarlo con lo de hoy

Un `deque<int>`: `N` es `front`, `pop_front` y `push_back`, todo $O(1)$. `E x` busca a x con un iterador, lo borra si está y lo mete con `push_front`. Con a lo sumo C atenciones, solo pueden llegar al frente los primeros $\min(P, C)$ y los que se adelanten. Cada caso cuesta $O(C \cdot \min(P, C))$, a lo sumo del orden de 10^6 pasos.

Las trampas

- Llenar la fila con los P ciudadanos: con $P = 10^9$ no cabe en memoria ni en tiempo.
- `E x` con x ya en la fila no lo duplica: sale de su puesto antes de pasar al frente.
- x puede ser mayor que C y no estar en lo guardado. Entonces solo se pone al frente.

Contenido I

1 Lo que ya está construido

2 `list`: la lista doblemente enlazada

3 `deque`: la cola doble que se lee por índice

4 `stack` y `queue`: adaptadores

5 Cuál escoger

6 Medir

7 Para el juez

8 Ejercicios

Ejercicio en parejas: los dos extremos I

Escriban `bool palindromo(const char *s)` con un `deque<char>`: la palabra se lee igual al derecho y al revés.

Las preguntas

1. ¿Qué se compara en cada vuelta y qué se saca?
2. ¿Cuándo se detiene, con longitud par y con longitud impar?
3. ¿Cuánto cuesta en tiempo y en espacio aparte? ¿Serviría una queue?

Ejercicio en parejas: los dos extremos II

```
bool palindromo(const char *s) {
  deque<char> d;
  int i = 0;
  while (s[i] != '\0') {
    d.push_back(s[i]);
    i = i + 1;
  }
  bool iguales = true;
  while (iguales && d.size() >= 2) {
    if (d.front() == d.back()) {
      d.pop_front();
      d.pop_back();
    } else {
      iguales = false;
    }
  }
  return iguales;
}
```

Ejercicio en parejas: los dos extremos III

Lo que debe salir

Se comparan `front` y `back` y, si coinciden, se sacan los dos. Se detiene al quedar cero o un carácter, o en el primer par distinto: `reconocer`, `radar`, `a` y `abba` dan `si`; `lista` y `abca`, `no`. Cuesta $\Theta(n)$ en tiempo y $\Theta(n)$ en espacio aparte, el `deque`. Una `queue` no sirve: no deja sacar por el final.

Ejercicio en parejas: el máximo de cada ventana I

Dado un arreglo $a[0..n-1]$ y $k \leq n$, imprimir el máximo de cada ventana de k casillas seguidas: $a[0..k-1]$, $a[1..k]$, y así hasta $a[n-k..n-1]$.

Las preguntas

1. Con $a = \langle 4, 2, 12, 3, 8, 7, 1, 5 \rangle$ y $k = 3$, ¿qué se imprime?
2. Escriban la versión directa. ¿Cuánto cuesta?
3. Si $a[i] \leq a[j]$ con $i < j$, ¿puede $a[i]$ volver a ser el máximo de una ventana que contenga a j? ¿Qué se puede botar entonces?

Ejercicio en parejas: el máximo de cada ventana II

```c
// Theta((n - k + 1) k): cada ventana se recorre entera
void maximosDirecto(int a[], int n, int k) {
  int j = k - 1;
  while (j < n) {
    int m = a[j - k + 1];
    int i = j - k + 2;
    while (i <= j) {
      if (a[i] > m) {
        m = a[i];
      }
      i = i + 1;
    }
    printf(" %d", m);
    j = j + 1;
  }
  printf("\n");
}
```

Ejercicio en parejas: el máximo de cada ventana III

- Imprime `12 12 12 8 8 7`. Hay $n - k + 1$ ventanas y cada una se recorre entera: $\Theta((n - k + 1)\, k)$, que con $k = n/2$ es del orden de n^2. Espacio aparte $\Theta(1)$.

Ejercicio en parejas: el máximo de cada ventana IV

```
void maximosDeque(int a[], int n, int k) {
  deque<int> d;          // indices; valores decrecientes
  int j = 0;
  while (j < n) {
    while (!d.empty() && a[d.back()] <= a[j]) {
      d.pop_back();            // ya no puede ser maximo
    }
    d.push_back(j);
    if (d.front() <= j - k) {
      d.pop_front();           // salio de la ventana
    }
    if (j >= k - 1) {
      printf(" %d", a[d.front()]);
    }
    j = j + 1;
  }
  printf("\n");
}
```

Ejercicio en parejas: el máximo de cada ventana V

Lo que sostiene el deque

Sea $d = \langle i_1, \ldots, i_m \rangle$ el contenido al terminar la vuelta j, con $j \geq k - 1$.

1. $\forall t,\ 1 \leq t \leq m :\ j - k < i_t \leq j$.
2. $\forall t,\ 1 \leq t < m :\ i_t < i_{t+1} \ \wedge\ a[i_t] > a[i_{t+1}]$.
3. $a[i_1] = \text{máx}\{a[r] : j - k < r \leq j\}$.

- La respuesta a la pregunta 3 es no: $a[j]$ le gana a $a[i]$ y se queda más tiempo en la ventana. Por eso el `while` bota del final a los que no superan a $a[j]$.
- Cada índice entra una vez con `push_back` y sale a lo sumo una vez: $\Theta(n)$ en total. Espacio aparte $O(k)$, el `deque`. La salida coincide con la directa.

Propuestos para la casa I

1. **Rotar.** Escriba `rotar(d, k)`, que pasa los k primeros elementos de un `deque` al final, en el mismo orden. ¿Cuánto cuesta? ¿Y si fuera un `vector`?
2. **Intercalar.** Dadas dos `list<int>` `a` y `b`, deje en `a` un elemento de cada una, alternando, sin crear nodos nuevos. ¿Qué operación de la lista mueve un solo nodo?
3. **Una pila sobre `list`.** Declare `stack<int, list<int> >` y repita los paréntesis balanceados. ¿Cambia algún costo? ¿Cambia la memoria?
4. **El historial con `vector`.** Resuelva el primer programa del ejercicio de decidir con un `vector` y mida con 10^5 visitas. Compare con el `deque`.
5. **Ventana mínima.** Cambie `maximosDeque` para que imprima el mínimo de cada ventana, y escriba los tres invariantes que lo sostienen.

Pistas para los propuestos I

1. Cada paso es `push_back(front())` y `pop_front()`: $O(k)$. Con `vector`, cada salida por el frente corre todo.
2. `splice` tiene una versión que recibe un iterador a un solo nodo de la otra lista y lo mueve en $\Theta(1)$. Guarde el iterador al siguiente antes de moverlo.
3. Los costos asintóticos son los mismos; la memoria pasa de bloques a un nodo con dos punteros por elemento.
4. Tome el reloj antes y después, como en la medición de insertar al frente, y duplique el número de visitas para ver cómo crece.
5. Solo cambia la comparación del `while`; en los invariantes, $>$ pasa a $<$ y el máximo a mínimo.

Resumen I

- `list` es la lista doblemente enlazada: inserta, borra y empalma con `splice` en $\Theta(1)$ con el iterador, no invalida iteradores al insertar y no tiene índice. Se ordena con `l.sort()`, no con `sort`.
- `deque` es la cola doble con índice: bloques fijos y un mapa de punteros dan los extremos y `d[i]` en $O(1)$; insertar en medio corre el lado más corto.
- `stack` y `queue` son adaptadores sobre un `deque` por defecto; el contrato limita las operaciones, y el contenedor de abajo se cambia si da lo que el adaptador pide.
- Las mediciones confirman las tablas: el `vector` crece como n^2 al insertar por el frente, y leer la `list` por posición también.

Resumen II

Lo que sigue

Se terminan los contenedores con lo que falta de ellos y se abre la tabla de direccionamiento directo: guardar cada elemento en la casilla que dice su clave, para buscar en $O(1)$ sin recorrer.

Referencias I



cppreference.com. Containers library, con la complejidad de cada operación: `https://en.cppreference.com/w/cpp/container/list`, `https://en.cppreference.com/w/cpp/container/deque`, `https://en.cppreference.com/w/cpp/container/stack`, `https://en.cppreference.com/w/cpp/container/queue`, `https://en.cppreference.com/w/cpp/container/priority_queue`.



T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022. Capítulo 10, estructuras de datos elementales: §10.1 arreglos, matrices, pilas y colas; §10.2 listas enlazadas.



R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulos 6, listas enlazadas; 7, pilas; y 8, colas, incluida la cola doble.